

Explainable Network Verification via Localized Subspecification

Paper #570, 12 pages body, 17 pages total

ABSTRACT

Network verification, synthesis, and repair tools help enforce high-level operational intent, but their limited explainability makes configuration maintenance costly in practice, as operators must still manually reason about large, low-level configurations. We propose *localized subspecifications*, which explain how individual configuration elements preserve a given network property by constraining their admissible behaviors. A user study with 15 professional network operators and 8 graduate students shows **52%** higher accuracy and **24%** time saving, and **70%** participants report they would like to use subspecifications in daily operations, demonstrating practical benefits. To support real deployments, we develop a scalable explanation generation algorithm with soundness guarantee. It takes **21** minutes on real-world Internet2 configuration, and **81** minutes on networks with up to **500** routers.

1 INTRODUCTION

Network operators manage a network by enforcing a set of *high-level* properties, or specifications, *e.g.*, reachability, isolation, and routing path preference [4, 10, 19, 25], with a set of *low-level* configurations, *e.g.*, route maps, prefix lists, and attribute manipulations. It depends on the network control plane, *e.g.*, BGP and OSPF, to interpret the configurations, and derive the network state which satisfy the specifications. Such a *semantic gap* between low-level configuration and high-level specification is a long-standing source of configuration mistakes and operational overhead, especially during network maintenance and updates [11, 13, 15].

A broad range of automated tools address this semantic gap across different operational settings. Network verifiers [5, 10, 25] check whether a given configuration C satisfies a specification φ . Synthesis tools [11, 12, 17, 18] construct brand new configurations that satisfy a given specification, while repair tools [22] generate patches to a configuration that violates the specification.

Despite significant progress in automated tools, their adoption in production networks is still low. We argue a key reason is due to the lack of *explainability*, a topic largely underexplored in the community. Modern network configurations are large and complex, making it difficult for operators to fully understand the intricate connections between configurations and specifications. Without such an understanding, and considering the critical nature of network operation, it

is hard for operators to trust and apply the results of verification, synthesis, or repair tools.

For example, consider a policy requiring that no private IPv4 prefixes within the 10.0.0.0/16 block is advertised externally. Figure 1 shows two policy implementations using prefix-list. Both could pass the verification against the intent, but the second is overly restrictive: it blocks all prefixes under 10.0.0.0/8 rather than just those under 10.0.0.0/16, unintentionally suppressing legitimate internal routes. This motivates explainable verification methods that not only validate correctness but also reveal which configuration choices are *actually required* to enforce the intended semantics.

permit 10.0.0.0/16 ge 16 permit 10.0.0.0/8 ge 8

(a) Exact.

(b) Overly restrictive.

Figure 1: Two prefix-lists for private IPs in 10.0.0.0/16.

Therefore, we envision a tool with fine-grained explanation for configurations can be quite useful for operators to connect high-level specification with low-level configuration, thereby paving the way for adoption of automated tools.

A natural way seems to leverage the code explanation features powered by LLMs. However, LLM-based code explanation does not guarantee correctness. A misled explanation can cause operators to adopt an erroneous configurations. Secondly, the current LLMs still has a limited context, and fail to capture the intricate dependency among a huge number of configuration lines on thousands of devices. **For example, in our user study (Sec. 6), a professional operator used an AI-based assistant to interpret a route-map line. Although the assistant correctly explained the line’s local semantics, it failed to expose a downstream dependency, leading the participant to approve an unsafe update.**

Key idea: precise configuration explainability via subspecification. We approach explainability via *localized subspecification*: the precise requirement that each configuration element must satisfy for the global specification to hold, assuming rest of the configuration remain *fixed*. This localization **reduces** reasoning about large, complex networks to focused, per-element analysis, making explicit how each configuration choice contributes to high-level operational intent. Returning to the example in Figure 1, the line h : permit [p] ge [l] should have the following subspec:

$$\psi(h) : \text{Addr}(p) \supseteq 10.0.0.0/16 \wedge l \leq 16 \quad (1)$$

That is, as long as the prefix parameter p covers all addresses in 10.0.0.0/16 and the ge parameter $l \leq 16$, the policy

remains satisfied. It reveals that the line in Figure 1b is overly restrictive, as it enforces conditions stronger than required.

Conceptually, localized subspecifications (subspec) are analogous to *pre/post conditions* in program verification: precisely capturing the behavioral contract at each configuration location. This contract-based view enables modular reasoning, clearer explanations, and safer configuration evolution.

Compared to LLM-based code explanation, subspecifications has a number of merits: (1) it specifies what a line *must satisfy* to preserve correctness, rather than its current behavior; (2) it provides precise and sound explanations without hallucination; and (3) it can be computed efficiently (Sec. 7).

Use cases. Localized subspecifications act as an *explanation layer* that augments existing verification, repair, and synthesis tools, helping operators understand and safely modify configurations during routine maintenance.

Explaining large scale configurations. Localized subspecs can identify the small set of lines critical to a given property, e.g., $\approx 0.7\%$ lines in Internet2 (Sec. 7). This guides attention to route-relevant locations, supporting scalable debugging, maintenance, and refactoring.

Auditing repairs and updates. When configs changes (by manual or automatic repair), operators need to know not just if a patch is correct, but why. Localized subspecs expose the admissible behavior at each modified location, making fix rationales explicit and enabling confident auditing.

Understanding synthesized configuration. Synthesis tools may generate correct yet unintuitive configs or programs [9, 23]. Localized subspecs expose how each element is constrained and how the config satisfies the global specification.

Challenges. Deriving precise subspecifications, *i.e., without assuming a fixed surrounding configuration*, is challenging because configuration elements interact through routing protocols, and many global configurations may satisfy the same specification. Isolating the exact contribution of any single element therefore requires complex global reasoning, and remains a non-trivial manual effort in practice [6, 28].

By fixing the configuration and analyzing only one location at a time, we avoid exploring alternative implementation at other locations, making automatic reasoning tractable. Localization also implies non-compositionality, *i.e.*, individually safe updates may interact when combined (Proposition 5), but localized subspecifications remain highly effective for configuration maintenance.

Contributions and organization. This paper tackles these challenges after reviewing network verification (Section 2) and presenting an illustrative example (Section 3).

Subspecification formulation (Section 4). We define valid configuration locations at multiple granularities and formalize the syntax and semantics of localized subspecifications.

Subspecification generation algorithm (Sections 5). We present a scalable algorithm that decomposes global reasoning into router-local analysis based on control-plane stable state. We establish its correctness, conciseness, and parallel scalability.

User study (Section 6). In task-based studies with network operators and researchers, subspecifications improve accuracy by 52% and reduce completion time by 24%. 70% participants would like to use our tool frequently. The study tasks are provided in supplementary materials [7].

Evaluation (Section 7). On real-world and synthetic configurations, subspec generation is efficient and scalable: 21 minutes on Internet2, and 81 minutes on 500-router network.

We conclude by discussing the limitations and fundamental tradeoffs of localized subspecifications.

Ethics statement. This work includes a user study with informed consent and no sensitive data. It followed institutional and community guidelines and received approval from the lead author’s institution. Participants were allowed to withdraw at any time, and were debriefed after study.

2 BACKGROUND AND MOTIVATION

We begin by outlining why network configurations are difficult to interpret, discuss limitations of existing approaches, and then motivate our design of localized subspecifications.

Network configurations are hard to read. Configurations are hard to reason about for many reasons. In particular, we highlight: (1) *Semantic subtlety*: similar commands can have subtle difference across vendors; (2) *scale*: long prefix-lists make IP prefix matching hard to compute mentally; (3) *inter-dependencies*: interactions across protocols and routers are complex; and (4) *global reasoning*: alternative convergence of routing protocols are hard to reason about.

Automated reasoning. To cope with this complexity, various automatic reasoning tools have been developed.

Verification. Verification tools check if a configuration satisfies desired properties. Symbolic methods [10] encode protocol semantics and policies as logical constraints over routes and explore feasible stable states via constraint solving, providing strong guarantees but limited scalability. Simulation-based methods [19, 25, 32] instead compute a converged stable state (e.g., a forwarding graph) and check properties directly on it. There are typically more scalable and forms the basis of our stable-state-driven decomposition.

Synthesis and repair. Similarly, synthesis and repair techniques treat parts of the configuration as unknowns and solve for

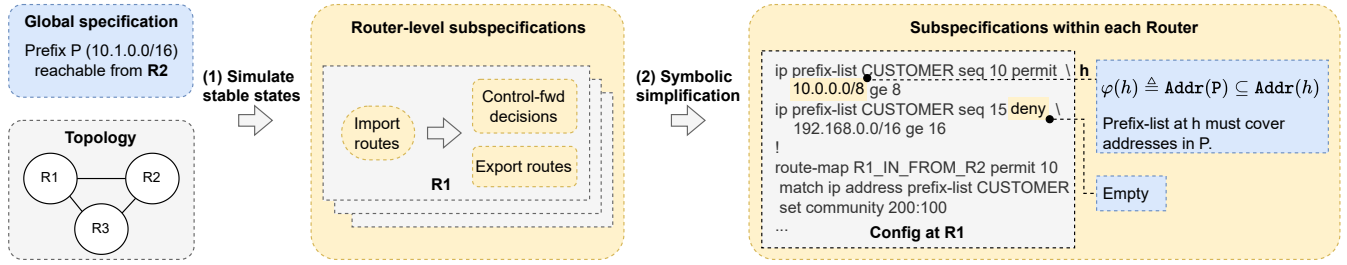


Figure 2: Subspecification generation flow, using stable control-plane behavior as router-level subspecification.

Approach	Configuration	Routes
Verification	Fixed	Computed
Synthesis	Computed (global, single)	Fixed
Localized Subspec	Computed (local, all)	Fixed

Table 1: Conceptual contrast with existing work.

assignments that satisfy routing constraints [4, 11, 12, 17, 18, 26], often relying on verifiers to validate candidate solutions.

Explainability: a missing part in network verification.

As reported by participants in our user study (Sec. 6), despite advances in automatic network reasoning, manual inspection of configurations remains unavoidable in practice. This is partially due to the high-stakes nature of network and imperfect modeling in these tools. The other reason, we argue, is the lack of explainability of the verification results.

Local subspecification for explainable verification. We use *localized subspecifications* as an explainability mechanism, which associates each configuration element with constraints on the edits that preserve a verified property under the current stable state. Table 1 highlights the contrast: unlike verification (configs fixed, routes computed) and synthesis (single config computed globally), localized subspec fix the stable routes and compute *all admissible local configuration implementations*, e.g., Eq. 1, turning global correctness into localized explanations.

Unlike provenance-style explanations [16, 35, 36] (why behavior occurred) or global intent inference [13, 21] (what the network as a whole enforces), subspecs give *local, normative explanations*: what each element must satisfy to remain correct. It addresses explainability along two dimensions: *Exposing modeling assumptions.* Subspecs explicitly reveal the assumptions made by the underlying verifier, clarifying subtle vendor-dependent semantics, e.g., whether `le 32` is interpreted as inclusive, thereby improving transparency.

Reducing reasoning burden. (1) *Semantic clarification*: subspecs make config semantics explicit, reducing ambiguity; (2) *Attention focus*: only a small fraction of lines are relevant for routes to a given prefix, e.g., $\approx 0.4\%$ fields in Internet2 (Sec. 7); (3) *Intuition validation*: subspecs expose overlooked cross-router dependencies when operator’s intuition judgments

Symbol	Meaning
C	Network configuration.
φ	Global specification / correctness property.
$S(C)$	Stable control-plane state (routes) of configuration C .
h	A specific location in the configuration.
$\psi(h)$	Localized subspecification at location h .
$C[h \leftarrow g]$	Configuration C with location h replaced by g .

Table 2: Summary of notations.

conflict with subspec (Sec. 6); (4) *Local reasoning*: operators can check edits against local constraints without mentally simulating global convergence.

3 OVERVIEW

Figure 2 shows the generation flow of *localized subspecs*.

Goal. Given a configuration C that satisfies specification φ ($C \models \varphi$), derive for each location h (e.g., a prefix-list line or parameter) a constraint $\psi(h)$ capturing the minimal requirement on h to preserve φ . Equivalently, $\psi(h)$ characterizes all replacements g at h such that $C[h \leftarrow g] \models \varphi$.

Table 2 summarizes the notations.

Weaker guarantee for tractability and interpretability.

Computing subspecifications is as hard as full network verification, as the impact of each location depends on global routing convergence (Sec. 5.1), and must be repeated across a large number of locations (Table 3).

To make the problem tractable, we relax the goal. Rather than explaining why a configuration C satisfies a specification φ under *all* possible converged states, we focus on a more concrete question: why does C converge to the *current stable control-plane state* $S(C)$ that satisfies φ ? In other words, subspecifications explain the correctness of the observed stable state, not all hypothetical alternatives.

This relaxation intentionally trades completeness for (1) *efficiency*: it enables a two-phase, router-local analysis, yielding a scalable algorithm with soundness guarantee and parallel scalability (Sec. 5.4); and (2) *interpretability*: operators reason about concrete, realized routing behavior, rather than mentally simulating alternative convergence outcomes.

The subspec generation proceeds in two phases.

Location	Signature	Interpretation
<i>Assume-guarantee functional subspecifications</i>		
Router	$(IF \rightarrow Route) \Rightarrow (Route, IF \rightarrow (Route, Bool))$	imported routes per interface, to a best route and export decisions per interface.
Route-map	$Route \Rightarrow (Bool, Route)$	input route, to permit/deny decision and updated route.
<i>Predicate (range) subspecifications</i>		
Line	$Field^* \Rightarrow Boolean$	Constraint on admissible combinations of its fields.
Field	$Field \Rightarrow Boolean$	Constraint on the field's admissible values.

Table 3: Subspec forms across config granularities.

(1) Router-level subspecification (Fig. 2, middle). From the simulated stable control-plane state we extract for R1:

- routes imported from R2 and R3 toward **10.1.0.0/16**;
- a best-path selection that prefers the route via R2;
- the requirement that exported routes must not carry the community tag NO_TRANSIT, since such routes would be denied by downstream routers.

Together, these form the router-level subspecification, which captures the minimal requirements at R1 needed to sustain the original stable state (Section 5.2). Because this stable state has already been validated by verification, preserving it is sufficient to maintain correctness.

This router-level subspecification explains how φ is satisfied: R1 forwards all traffic to **10.1.0.0/16** via R2. It also localizes subsequent reasoning: field-level subspecification only needs to identify which configuration fields at R1 preserve this *local behaviors*.

(2) Field-level subspecification (Figure 2, right). For each location h , we use a symbolic encoding and simplification approach to derive the condition under which the router continues to satisfy its required imports, best-route choice, and exports. This condition forms the field-level subspecification $\psi(h)$. **Here the subspec at the prefix-list COSTUMER seq 5 10.0.0.0/8 field requires that it match all addresses in the target prefix P (10.1.0.0/16). For the seq 10 deny field, the subspecification is empty, indicating that the field may take any value without violating reachability.** See Section 5.3 for details and generalization to larger blocks.

In this process, field-level subspecifications directly connect the *concrete configuration* with the *abstract router-level behavior*. Each $\psi(h)$ captures the minimal condition that a specific configuration field must satisfy, explaining precisely *why* that line matters and filtering out irrelevant details.

Scope and limitation. This formulation explains properties expressible as constraints on the network’s stable control-plane state [10], e.g., reachability and path preference, and

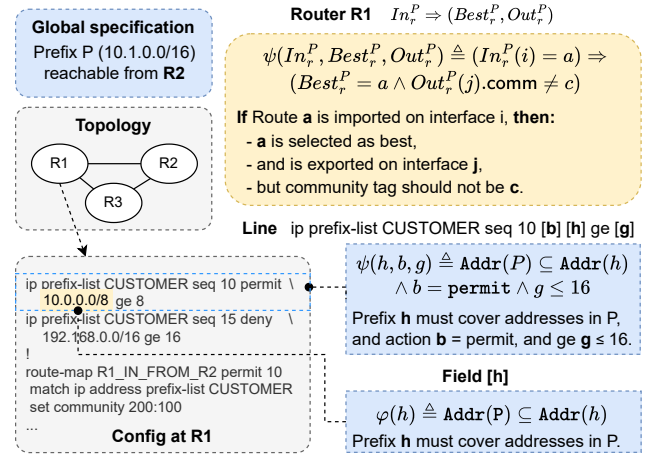


Figure 3: Subspecification at different granularities.

therefore assumes control-plane convergence. Its precision is defined with respect to the underlying verifier: edits that satisfy a subspecification are guaranteed to preserve the verifier’s conformance result, while inaccuracies in verifier’s model may lead to imprecise explanations. It does not cover transient behaviors [27] or performance [8].

Two limitations follow. First, subspecifications are *non-compositional*: each $\psi(h)$ assumes all other locations are fixed, so edits that individually satisfy their subspecifications may jointly violate φ (Proposition 5). Second, fault-tolerance behavior is out of scope, as it requires reasoning over alternative failure scenarios rather than a single stable state.

Nevertheless, subspecifications remain highly useful: they complement verification, synthesis, and repair tools, including LLM-generated configurations, by serving as post-hoc correctness contracts to validate and audit changes.

4 SUBSPECIFICATION

We formalize localized subspecifications by first identifying valid configuration locations at different granularities, defining the corresponding subspecification formats for each level, and then illustrating them with concrete examples.

Location in a network configuration. A *location* h denotes a structural unit in a router configuration. As summarized in Table 3, locations range from entire routers and route-maps down to individual lines and fields.

Subspecification forms must align with the semantics of the configuration elements they explain. High-level constructs such as routers and route-maps define *functional behavior* over routes, while low-level configuration elements encode *parameter choices* within given structure. Accordingly, we adopt two complementary subspecification forms.

(1) *Assume-guarantee contracts.* For routers and route-maps, subspecifications take a functional assume-guarantee form, describing how admissible inputs (*i.e.*, imported routes) constrain outputs (*i.e.*, best-route selection and export routes).

(2) *Value constraints.* For lines and fields, subspecifications are predicate-style constraints that restrict the admissible parameter values, such as prefix ranges or attribute bounds.

Semantics. Given a configuration C that satisfies a global network property φ (*i.e.*, $C \models \varphi$) and a location h , the *localized subspecification* at h is a constraint $\psi(h)$ such that $\forall g. g \models \psi(h) \iff \varphi(C[h \leftarrow g])$. That is, $\psi(h)$ precisely characterizes the set of local instantiations at h that preserve φ , assuming the remainder of the configuration $C \setminus h$ is fixed.

Intuitively, any update at location h that satisfies $\psi(h)$ is guaranteed to preserve correctness, while violating $\psi(h)$ may break the global property. By explicitly characterizing this *viable range* of safe edits, localized subspecifications explain why a configuration element must take its current form and how it contributes to the global intent.

Relaxation. As discussed in Sec. 3, reasoning over all possible control-plane steady states is expensive and often produces large, hard-to-interpret subspecifications. We therefore relax the problem by requiring preservation of the same stable state only (Sec. 5). This yields shorter, more readable subspecifications and enables scalable generation (Sec. 7).

Example 1:[Subspec at different granularities] Figure 3 illustrates how a global network property is explained through localized subspecification at different granularities. The specification φ requires prefix P to remain reachable from router $R2$. Under the configuration C , router $R1$ imports a route a to P , selects it as the best route, and exports it to peers.

Router level subspecification at $R1$ requires that if it receives route a for P , it must continue to select and export a , and avoid attribute changes (*e.g.*, community filters) that would prevent downstream reachability. This explains not only what the router does, but the exact *set* of alternative behaviors would still preserve the same stable state.

At finer granularity, this reduces to simple parameter constraints: the prefix h must cover P , action $b = \text{permit}$, and the ge parameter g must satisfy $g \leq 16$. Any edit within this range preserves the stable state and reachability. $\square$

5 SUBSPECIFICATION GENERATION

We formulate the subspecification generation problem, present a two-phase algorithm that balances completeness and scalability, and establish key properties of the algorithm.

5.1 Problem Formulation and Challenges

We first revisit the challenge of deriving localized subspecification in its original definition in Section 4.

Challenge. Deriving an *iff* (logic equivalence) localized subspecification *w.r.t.* to the global property φ is as hard as full network verification. A local configuration change can alter control-plane convergence and route propagation across the entire network, requiring global reasoning to determine whether φ still holds. Since full verification is computationally expensive for large networks, repeating it independently for each of N configuration locations is prohibitively costly (See experimental comparisons in Section 7).

Example 2:[Update-induced global reasoning] Router R_1 reaches prefix P via neighbor R_2 (primary path) and has a backup path via R_3 . A route-map at R_1 is modified to add a community filter that causes the route from R_2 to be dropped. As a result, R_1 selects the backup route via R_3 , changing the stable control-plane state (different next hop and AS-PATH), while end-to-end reachability to P still holds. Hence determining whether reachability still holds requires re-analyzing a new control-plane convergence. $\square$

We therefore adopt a relaxed formulation that enables router-local decomposition and scalable generation.

Relaxed formulation. We relax the notion of subspecification based on the *observed stable control-plane state*.

Input. (1) A global network property φ ; (2) A configuration C that is verified to satisfy φ ;

Output. A set of localized subspecifications $\Psi = \{\psi(h) \mid h \in \text{Loc}(C)\}$, where each $\psi(h)$ characterizes the set of configuration at location h that preserve the same stable state $S(C)$, assuming all other configuration elements remain unchanged. Preserving $S(C)$ in turn guarantees the configuration continues to satisfy the global property φ .

Based on this relaxed formulation, we use a two-phase approach: first extract router-level subspecifications via control-plane simulation, then derive field-level subspecifications from these summaries through symbolic simplification.

5.2 High-level Functional Subspec

We revisit router-level subspecifications and show how they are derived via control-plane simulation and analysis.

Router-level subspecification. As defined in Section 4, the router-level subspecification for a router r consists of the assumed imported routes, the admissible best-route selection, and the export decisions.

The *assume* part of the subspec is taken directly from the stable state, since r is guaranteed to import the same routes when configurations on all other routers remain fixed. The best route selection is likewise fixed to its stable-state value, as the subspec must ensure convergence to the same state.

Export decisions, however, allow flexibility: different export attributes may still lead to the same stable state.

```

1: procedure GenSubspec ( $S_r, C_r$ )
2:    $\Psi \leftarrow \emptyset$  ▷ init subspecification map
3:   for field  $h$  in  $C_r$  do
4:      $v \leftarrow$  fresh symbolic variable
5:      $\text{seed}(v) \leftarrow \text{ENCODE}(C_r[h \leftarrow v])$ 
6:        $\wedge \text{STATEEQ}(S_r)$ 
7:      $\psi_h \leftarrow \text{SIMPLIFY}(\text{seed}(v))$ 
8:     if  $\psi_h$  is trivially true then
9:       if  $\text{seed}(v) \wedge v \neq C_r[h]$  is UNSAT then
10:        mark  $\psi_h$  as a constant (its current value)
11:       end if
12:     end if
13:   end for
14: return  $\Psi$ 
15: end procedure

```

Figure 4: Field-level subspecification derivation.

Export attribute constraints. Export-attribute constraints are derived by analyzing how route attributes affect downstream routing behavior in the stable state. The goal is to ensure that a route continues to propagate along the same path as observed in the original stable state.

Specifically, for attributes used only for *filtering* (e.g., community tags), we derive just the constraints needed to preserve those filtering decisions, avoiding unnecessary restrictions. In contrast, attributes that affect route preference metrics, such as AS-PATH or OSPF cost, require cross-route comparisons and more complex reasoning; we therefore keep them fixed to their stable-state values.

This analysis proceeds backward along the propagation path. Starting from the last router, we derive attribute requirements that preserve its route filtering decision and propagate these constraints upstream. The result is a minimal set of export-attribute constraints at the source router that is sufficient to maintain the same stable routing outcome.

Example 3:[Export constraints] Consider a propagation path $R_1 \rightarrow R_2 \rightarrow R_3$ for prefix P under a no-transit policy. In the stable state, R_3 blocks export of routes with `community = NO_TRANSIT`, R_2 forwards routes unchanged, and R_1 sets this community on export toward R_2 .

Starting from R_3 , the blocking behavior from the original stable state requires its import route from R_2 to carry `community=NO_TRANSIT`. Since R_2 does not modify this attribute, the same requirement propagates to R_1 . The resulting export constraint at R_1 is $\psi_{\text{export}}(R_1)$: `community = NO_TRANSIT` $\wedge$ `other_attributes = stable-state_values`.

In contrast, suppose R_1 also sets an auxiliary community `MONITOR`. Since no downstream policy matches or depends on this tag, it does not appear in any propagated constraint and is therefore unconstrained in the subspecification. $\square$

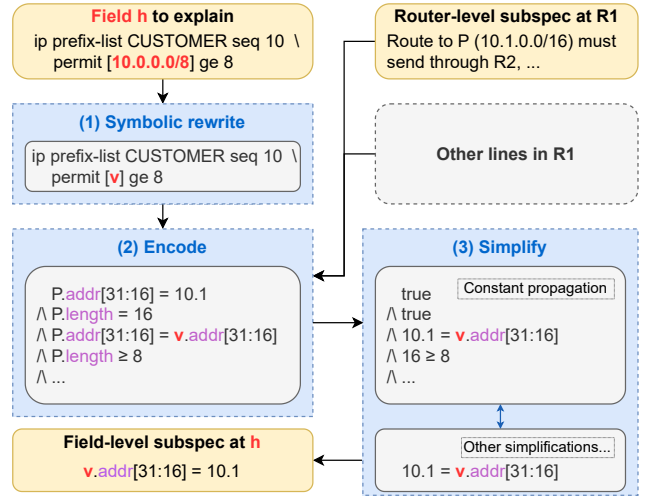


Figure 5: Example of field-level subspec derivation.

Let $\psi_{\text{export}}(r)$ be the export-attribute constraint derived for a route to prefix P along its stable propagation path.

Proposition 1:[Correctness of export-attribute constraints] For any modified configuration, if it satisfies $\psi_{\text{export}}(r)$ and all other configurations remain unchanged, then the stable control-plane state for P remains unchanged. $\square$

Proof sketch. We prove that every router on the stable propagation path p receives the same route and makes the same decision as in the stable state.

Base case. At the first router on p , metric-related attributes are fixed to their stable-state values and filtering attributes satisfy the derived constraint, so it exports the same route.

Induction. Assume R_i exports the same route as in the stable state. The constraint ensures this route passes R_{i+1} 's filters. Since preference-related attributes are unchanged, R_{i+1} selects the same best route and exports it.

Thus all routers on p behave as before, and the stable state for P remains unchanged. $\square$

5.3 Low-level Parameter Constraints

We next derive line/field-level subspecs that capture how individual parameters preserve the verified stable states.

Field-level subspecification problem. Given a router-level specification $S_r^P = \langle In_r^P, Best_r^P, Out_r^P \rangle$ as above, and the corresponding configuration C_r , we compute, for each location h in C_r , the localized subspecification $\psi(h)$ s.t.

$$\forall g. g \models \psi(h) \iff C_r[h \leftarrow g] \models S_r^P.$$

That is, an update at location h satisfies $\psi(h)$ iff the modified configuration preserves the router-level subspecification.

Algorithm. As shown in Figure 4, given router-level subspec S_r and configuration C_r , we generate field-level subspec

by replacing each field h with a fresh symbolic variable v and deriving constraints $\text{seed}(v)$ that preserve S_r . These are simplified via SMT rewriting, followed by a validation step to distinguish constant from irrelevant fields. The resulting formulas constitute the localized subspecifications.

Example 4:[Field-level subspec generation] Figure 5 illustrates the derivation process. Consider the prefix-list entry and a stable route with prefix $P = 10.1.0.0/16$.

```
ip prefix-list CUSTOMER seq 10 permit 10.0.0.0/8 ge 8
```

We replace the prefix with a symbolic variable v and encode the prefix-list semantics together with the router-level subspec constraints, yielding the seed subspecification:

$$\begin{aligned}\Phi(v) \equiv & P.\text{addr}[31:16] = 10.1 \wedge P.\text{length} = 16 \\ & \wedge P.\text{addr}[31:16] = v.\text{addr}[31:16] \wedge P.\text{length} \geq 8 \wedge \dots\end{aligned}$$

Applying simplification (Fig. 5, right) removes redundant constraints and produces the final field-level subspec:

$$\psi(v) \equiv v.\text{addr}[31:16] = 10.1$$

This constraint precisely characterizes all updates to the prefix field that preserve the stable routing behavior. $\square$

Constant vs. irrelevant fields. SMT simplification may eliminate a field either because its value is uniquely determined (*constant*) or because it has no effect on correctness (*irrelevant*). To distinguish these cases, when the subspecification at a location h simplifies to $\top$, i.e., unconstrained, we perform an additional validation step. Specifically, we check whether changing h can alter the router-level behavior: $(\text{seed}(v) \wedge v \neq C_r[h]) \models S_r$. If this check succeeds, then any deviation from the original value violates S_r , and h is classified as a constant. Otherwise, there exists an alternative value that preserves S_r , indicating that h is truly irrelevant and its subspecification is empty.

Example 5:[Irrelevant field] Consider a route-map line:

```
set local-preference 200
```

Suppose upstream policies ensure the same best route is chosen regardless of the local-preference value. Replacing the value with a symbolic variable v may let the solver simplify the constraint to $v = 200$ and eliminate v altogether.

This is ambiguous: v may be *constant* (200) or *irrelevant* (any value works). We therefore check if some $v \neq 200$ violates the verified behavior. If not, the field is irrelevant and its subspec is empty; otherwise, it is a true constant. $\square$

Line-level subspecification. We preserve the line's structure but treat its parameters symbolically, deriving joint constraints over these parameters as the line-level subspec.

Optimization: prefix-match scope analysis. To scale to large prefix-lists, we statically analyze, for a target prefix P , which rule l in the configuration it matches. Rules after l

are irrelevant (empty subspec), while preceding rules only need to preserve that they do not override l 's decision. **This prunes semantically irrelevant lines: instead of generating subspecs for every field in the list, we only compute them for the matching rule and one of the preceding rules.**

Example 6:[Prefix-list analysis] Consider a prefix-list:

```
seq 5 deny 10.0.0.0/8 le 8
seq 10 permit 10.1.0.0/16 ge 16
seq 15 permit 10.2.0.0/16 ge 16 <- matches P
seq 20 permit 0.0.0.0/0 ge 0
```

The target prefix $P = 10.2.0.0/24$ matches rule l at seq 15. Rules after l (seq 20) are never reached and receive empty subspecs. Rules before l (seq 5, 10) share the same constraint: they must not match P with a conflicting action, so one representative subspec suffices. Only the matching rule (seq 15) and first rule (seq 5) require a full subspec derivation. $\square$

Aligning simulation and symbolic encoding. We discovered and resolved modeling inconsistencies between the simulator [19] and verifier [10]. See Appendix A.2 for details.

5.4 Properties

We establish correctness, characterize the completeness trade-off, and analyze compositionality and performance.

Lemma 2:[Local equivalence] Assume the control-plane encoding ENCODE in GenSubspec (Figure Algorithm 4) correctly captures router semantics, i.e., for any configuration C ,

$$C \models S_r^P \text{ iff } \text{ENCODE}(C) \wedge \text{STATEEQ}(S_r^P) \text{ is satisfiable.}$$

For any router r , prefix P , configuration location $h \in C_r$, and any replacement g at h , let $\psi(h)$ the subspecification produced by GenSubspec . Then

$$g \models \psi(h) \text{ iff } C_r[h \leftarrow g] \models S_r^P. \quad \square$$

In other words, any edit at h satisfying its subspecification preserves the stable-state behavior of router r for prefix P .

Proof sketch. For a location h , the algorithm constructs

$$\text{seed}(v) := \text{ENCODE}(C_r[h \leftarrow v]) \wedge \text{STATEEQ}(S_r^P),$$

where (by assumption) $\text{seed}(v)$ is satisfiable exactly for those replacements of h that preserve the router-level behavior S_r^P .

It then sets $\psi(h) := \text{SIMPLIFY}(\text{seed}(v))$. Assuming SIMPLIFY is equivalence-preserving, $\psi(h)$ and $\text{seed}(v)$ define the same set of satisfying assignments to v . Therefore, for any g , $g \models \psi(h)$ iff $\text{seed}(g)$ holds, which (by the encoding assumption) is equivalent to $C_r[h \leftarrow g] \models S_r^P$. $\square$

Proposition 3:[Soundness] Let $\psi(h)$ be the subspec at location h and ϕ the verified global property. If $g \models \psi(h)$ and other config elements remain unchanged, then $C[h \leftarrow g] \models \phi$. $\square$

Proof sketch. By Lemma 2, $g \models \psi(h)$ preserves the router-level stable behavior. By Proposition 1, router-level preservation implies global stable-state preservation. Since φ holds on that state, $C[h \leftarrow g] \models \varphi$. $\square$

Example 7:[Soundness] A specification requires reachability to $10.1.0.0/16$. In the stable state, router r forwards traffic to this prefix via eth1, captured by the router-level subspecification S_r^P . For a location h in a prefix-list at r , GenSubspec derives $\psi(h)$ such that traffic still exits via eth1.

If an update violates $\psi(h)$, router r may choose a different next hop, yet reachability to $10.1.0.0/16$ can still hold via another path. Thus, subspecifications are *sound*: satisfying $\psi(h)$ preserves the verified behavior, while violating $\psi(h)$ does not necessarily violate the global specification φ . $\square$

Proposition 4:[Non-completeness] *There may exist an update g such that $\neg(g \models \psi(h))$ yet $\varphi(C_r[h \leftarrow g])$ still holds.* $\square$

Proof sketch. Since $\psi(h)$ is derived to preserve a specific stable state S_r , it excludes alternative configurations that lead to a different but still correct stable state S'_r satisfying φ . Therefore, $\psi(h)$ guarantees safety but not completeness. $\square$

Proposition 5:[Non-compositionality] *Let h_1 and h_2 be two distinct configuration locations with corresponding subspecifications $\psi(h_1)$ and $\psi(h_2)$, derived independently with respect to the same router-level subspecification S_r^P .*

There is no general guarantee that simultaneous updates g_1 at h_1 and g_2 at h_2 satisfying $g_1 \models \psi(h_1)$ and $g_2 \models \psi(h_2)$ will jointly preserve S_r^P , nor the global specification. $\square$

Proof sketch. Each $\psi(h_i)$ assumes all other locations are unchanged. Updating h_1 and h_2 together breaks these assumptions, so their combined effect may alter routing behavior. Thus, satisfying $\psi(h_1)$ and $\psi(h_2)$ individually does not guarantee preservation of S_r^P . $\square$

Example 8:[Non-compositionality] Consider a prefix-list:

```
ip prefix-list PL seq 5 permit 10.0.0.0/8
ip prefix-list PL seq 10 permit 10.1.0.0/16
```

The target prefix $10.1.0.0/16$ is accepted by either line. Individually, each line's subspecification allows arbitrary changes, since the other line still matches the prefix. However, modifying both lines simultaneously can leave $10.1.0.0/16$ unmatched, changing the router behavior. $\square$

Normalization. Assuming the underlying solver's simplification tactics are equivalence-preserving and eliminate tautologies and implied conjuncts, we apply them to each seed constraint. Under this assumption, the resulting subspecification is in normalized form with no syntactic redundancy: cannot remove or simplify any remaining conjunct without changing its logical meaning.

Group	#	Experience	Background
Grad. students	8	0.5–6.5 yrs	Networking research
Net. operators	15	1.5–11 yrs	ISPs / enterprise networks

Table 4: Participant demographics.

ID	Level	#Prefix	#Line	Task	Specification
1	Easy	8	117	Update	ECMP Reachability.
2				Cleanup	Block private IPs.
3	Hard	89	289	Cleanup	Reachability.
4				Update	No transit routes.

Table 5: User-study tasks and configuration scale.

The figure shows a screenshot of a network configuration interface. The top part displays a list of configuration lines for 'ip prefix-list'. A specific line is highlighted, and a hover pop-up appears showing a 'Prefix ge range = 20-24'. The bottom part shows the raw symbolic constraint for the same line, which is '(>= VAR 20) AND (<= VAR 24)'. The 'ge' parameter field is highlighted in blue, indicating it is constrained within the range [20, 24].

Figure 6: Subspec UI as hover pop-ups. Top: contextual interpretation; bottom: raw symbolic constraint. Here ge parameter field is constrained within $[20, 24]$.

Example 9:[Normalization] Consider a prefix-list entry at location h with seed subspec $\Phi(h) := (P.addr[31:16] = 10.0) \wedge (P.addr[31:24] = 10.0.1)$, constraining P to $10.0.0.0/16$ and $10.0.1.0/24$, respectively. Since the second prefix is covered by the first, the second constraint is redundant and simplified away, yielding a normal form $\psi(h) := (P \subseteq 10.0.0.0/16)$. $\square$

Parallel scalability. The overall subspecification generation procedure is parallelly scalable. Specifically: (1) router-level subspecifications are obtained via standard control-plane simulation, with scalability comparable to existing tools [19]; and (2) field-level subspecifications are derived independently per configuration location, enabling full parallelism. Empirical scalability results are in Sec. 7.

6 USER STUDY

To evaluate the usefulness of localized subspecification, we conducted a small user study [7] and answer the following:

RQ1. Do subspecs help users improve accuracy and efficiency in different network maintenance tasks.

RQ2. Do users find subspecs intuitive and easy to use?

6.1 Tasks and Study Structure

Participants. We recruited 15 professional network operators from ISPs and enterprise networks, and 8 graduate students in networking research, [through industry and academic contacts](#). Participant demographics are summarized in Table 4.

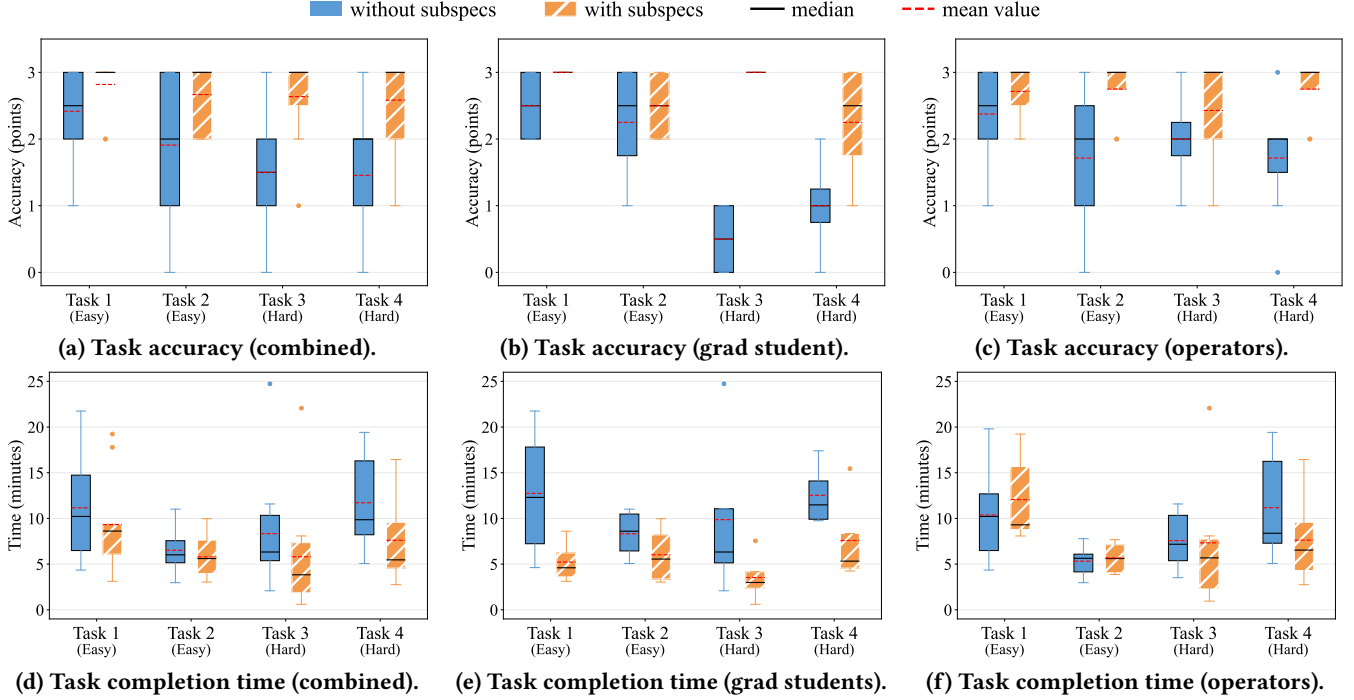


Figure 7: User study task accuracy and completion time.

Configuration and tasks. Table 5 summarizes four maintenance tasks and their configuration complexities. The configurations are designed to (1) reflect realistic operational challenges, while (2) remaining solvable within the one-hour study budget. We prepare two complexity levels, each with one refactoring task and one update task. For each task, three candidate edits are provided, and participants select those that preserve the specified properties.

Properties are enforced using three common constructs: prefix-lists for filtering, community tags for classification, and AS-PATH length for ECMP. This follows the engineering practice of using a small set of protocol attributes to simplify maintenance, as confirmed by our industry participants.

User interface. As shown in Figure 6, localized subspecifications appear as hover pop-ups attached to configuration blocks. Each pop-up shows an interpreted constraint at the top, where symbolic variables are instantiated with their configuration meaning, and the raw symbolic form below for exact semantics. Lines or fields with non-empty subspecifications are highlighted in a darker color (e.g., lines 9–12) to help users quickly identify semantically relevant elements. In this example, the target prefix matches line 12, while preceding lines are only required not to override line 12’s decision.

Control and treatment groups. Each task is shown in two conditions: a *control* condition with the plain configuration and a *treatment* condition with localized subspecifications. Other information are identical across both conditions.

To reflect realistic operational settings, participants in *both* groups are allowed to use *any* tool they typically use in practice, e.g., IP prefix conversion utilities or LLM-based assistants for clarifying configuration syntax and matching semantics (See further discussions below). This ensures that any observed improvements stem from subspecification-based explanations rather than tool restrictions.

Metrics. We consider two objective metrics: (1) *accuracy*, i.e., whether the selected option is correct, and (2) *completion time*, i.e., the time spent per tasks.

Workflow. The study begins with a short tutorial on the localized subspecification tool followed by several interactive warm-up questions to familiarize participants with subspecifications and the tasks. Participants then complete the tasks at their own pace. We record the accuracy and completion time for each user and task. After completing the tasks, participants fill out a System Usability Scale (SUS) questionnaire [14], a widely used 10-item Likert-scale instrument for measuring perceived usability and learnability of interactive systems. See Appendix B for the full questionnaire.

6.2 RQ1. Accuracy and Time Improvement

Accuracy. As shown in Figure 7a, (1) subspecs lead to more accurate configuration decisions overall. with the treatment group achieves on average 52% improvement over the control group; (2) gains are largest on complex tasks (3–4), where

long prefix-lists and richer interactions make manual reasoning error-prone; (3) without subspec, accuracy drops as task complexity increases, as expected; and (4) with subspec, accuracy remains stable, showing robust performance.

Time. As shown in Figure 7d, (1) participants with subspec are on average **1.3×** faster than those without (corresponding to a **24%** reduction in completion time); (2) the largest speedup appears in Task4 (**1.5×**), due to its higher complexity; (3) participants spend more time on Task1 for initial familiarization; (4) without subspec, completion time increases notably on harder tasks (3–4). (5) with subspec, time remains largely stable, with a slight increase on Task 4.

Impact of experience. Comparing graduate students and network operators (Figures 7b–7c, 7e–7f), (1) operators show smaller accuracy gains than students (**39% vs. 164%**), mainly due to a stronger baseline in the control group; and (2) operators didn’t save much time on simple tasks due to a more careful review of the configurations regardless of subspec, while graduate students improve consistently.

Impact of complexity. Network operators improve little on simple tasks (1–2) but gain clearly on harder tasks (3–4). As the study configurations are still smaller than real networks, the benefits are likely to grow at larger scales.

Anecdotes. Several anecdotes reveal interesting insights.

Subspec usage patterns. **We observe three frequent patterns.** (1) Many first jump to lines with non-empty subspecs using the visual cues (Fig. 6), rather than scanning long prefix-lists as in the control group, which is time-consuming. (2) When encountering unfamiliar vendor syntax, they use subspec to clarify the effective semantics assumed by the verifier, e.g., interpreting `le 32` as mask length ≤ 32 , resolving confusion without external lookup. (3) Some rely directly on subspecifications to judge edits without reading the full configuration, while others use them as a validator: when an intuitive judgment conflicts with a subspec, they revisit the relevant lines and often uncover overlooked dependencies.

Common cross-block errors. In the control group, errors often arose from large configurations and missed cross-element dependencies. For example, Task 3 includes long prefix-lists that require mentally tracing which entry a target IP matches; misjudgments here led to incorrect route classification and flawed downstream reasoning. With subspecifications, the matched and downstream-relevant lines are explicit via non-empty subspecification, directing attention to relevant lines.

Benefits beyond existing tools. Even equipped with IP prefix utilities or AI-based assistants, subspecs still led to measurable improvements. In our study, **7** participants used AI tools including Gemini [3], DeepSeek [1], and Doubao [2], supplying the full configuration and, when available, the

corresponding subspecs as their queries. Without subspecifications, AI assistants sometimes returned incorrect answers for the same tasks, whereas with subspecifications as additional context, the answers became correct. This indicates that subspecifications provide localized semantic constraints that are difficult to infer from configuration syntax alone.

Practitioners are more cautious. **With subspecification**, network operators spent **1.4×** more time reviewing the configuration on average, compared to graduate students. In post-study interviews, several of them noted that they preferred to fully understand the configuration before making any critical update, **due to the high-stakes nature of network operations and habits reinforced through training and practice**. Even so, subspecifications served as quick validators of their intuition, especially for unfamiliar syntax, thereby reducing the completion time.

6.3 RQ2. Usability Assessment

For space reasons, we summarize the main findings here and report detailed results and analysis in Appendix B.

SUS scores. Based on 23 responses: (a) 70% (16) would use the subspec tool in daily practice, while 9% (2) would not. (b) 52% (12) find it easy to use, only 4% (1) find it difficult. (c) Views on support are split: 52% (12) expect to need technical support, vs. 48% (11) who do not. Overall, subspecs show practical explanatory value beyond task-performance gains.

Qualitative feedback. Our post-study survey finds:

Positive themes. (1) Several participants saw explainability as a real workflow challenge and viewed subspecs as a promising way to understand configurations; (2) many said subspecs make it easy to sanity-check small updates without rereading everything; (3) the visual design, especially highlighting non-empty subspecs (Fig. 6), was often described as intuitive.

Limitations and concerns. (1) Some operators concerned about limited support for vendor-specific syntax. (2) Several expected more compositional explanations; for example, when a line has an empty subspec due to redundancy, they wanted to know which other line causes it. (3) The notion of subspecification was not immediately clear to a few participants, with some initially seeing it as a generic code explainer, though most understood it after the warm-up task.

These concerns point to both engineering and research directions. Broader vendor coverage and clearer onboarding can be addressed through improved modeling and UI design. In contrast, compositional explanations and richer dependency tracing suggest research on relating multiple subspecifications and exposing cross-location interactions.

Summary. (1) Subsolutions improve task accuracy by **52%** and reduce completion time by **24%** on average. (2) Gains

are larger on complex tasks and for less-experienced users (students vs. operators). (3) 70% of participants would use a subspec-based explainer in practice, vs. 9% who would not.

7 EVALUATION

We aim to answer the following research questions:

RQ3. Can subspecification be generated efficiently?

RQ4. Does it scale with network size and config complexity?

Baselines. We implemented our subspecification algorithm SubSpec in 7769 lines of Python and extended Batfish [19] and Minesweeper [10] in 3419 lines of Java. We compare it against two baselines: (1) FullSym, which **directly simplifies encoding of the full network to obtain localized subspecifications, without router decomposition**; and (2) SubSpec_{NS}, a variant of SubSpec without prefix match scope analysis, therefore has to analyze all config fields.

Configurations. To evaluate RQ3, we use real-world Internet2 configurations¹, and three synthetic configurations (Bics, Columbus, and USCarrier) from Config2Spec [24], which are generated from real-world network topologies². Table 6 summarizes the configuration characteristics.

To evaluate RQ4, we use large-scale synthetic FatTrees configurations from NV [20] and our own synthesized configurations, varying the number of configuration lines (prefixes) on a single device to assess worst-case per-device complexity.

Setup. For each configuration and target prefix, we first simulate the stable state with Batfish [19] and cross-check it using Minesweeper’s symbolic encoding [10] to calibrate modeling accuracy (discussed in Appendix A). We then generate subspecifications for all configuration locations (fields and lines) using SubSpec and the two baselines. Reported runtimes for SubSpec and SubSpec_{NS} include routing simulation and other preprocessing. We set a 4-hour timeout. Experiments ran on a 20-core 3.7GHz machine with 256GB memory.

Results. We summarize the main findings below.

RQ3: Efficiency. As shown in Figure 8, (1) on Internet2, SubSpec takes 21 minutes (avg. 14 ms per location), showing practical effectiveness; (2) on three synthetic configs, SubSpec takes 12-30 seconds (avg. 11 ms per location); and (3) across all configs, SubSpec consistently outperforms two baselines. FullSym times out on all but Bics; on Bics, SubSpec is 1.1× faster than SubSpec_{NS} and 293× faster than FullSym.

¹We exclude IS-IS configurations unsupported by Minesweeper, and a small number of lines that Batfish could not parse.

²OSPF-only configs are omitted since they mainly encode topology and link weights, with little routing-policy logic to explain.

Network	Type	#Routers	#Locations	%Subspec
Internet2	real-world	10	88534	0.4%
Bics	synthetic	49	1130	7.6%
Columbus	synthetic	86	1672	6.3%
USCarrier	synthetic	174	2470	5.4%

Table 6: Dataset characteristics.

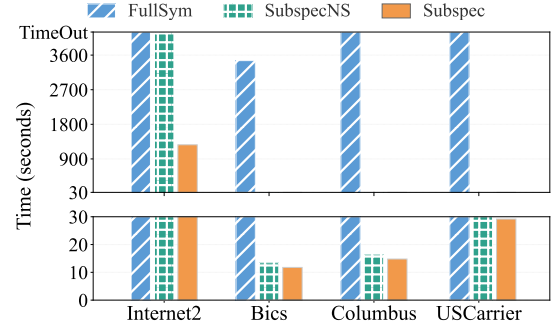


Figure 8: Run time on each network configuration.

Ratio of non-empty subspecification. As shown in Table 6, only 0.4 – 7% of configuration locations have non-empty subspecifications. This low subspec-to-location ratio means only a small fraction of the configuration requires attention, indicating that subspecifications help reduce cognitive load.

Optimization correctness. For all configs and locations, SubSpec and SubSpec_{NS} produce equivalent subspecifications, validating the correctness of prefix-scope optimization.

RQ4: Scalability. We assess the scalability of SubSpec by varying the number of routers, prefixes and threads.

Vary network sizes. As shown in Figure 9a, when the number of network devices increases from 20 to 500, (1) The runtime of SubSpec increases from 10 seconds to 81 minutes; (2) FullSym times out beyond 20 devices; and (3) SubSpec is 1.3× faster than SubSpec_{NS} on average.

Vary #prefixes. Figure 9b shows that as prefixes grow from 10 to 10,000, SubSpec runs in 4 seconds to 179 minutes, while SubSpec_{NS} and FullSym time out beyond 5,000, highlighting the benefit of prefix-scope analysis.

Vary #threads (Parallelism) (Figure 9c). On FatTree (180 devices), when threads increases from 1 to 20, (1) SubSpec and SubSpec_{NS} are 8× and 9× faster, respectively; (2) The main bottleneck is preprocessing, especially control-plane simulation, while subspec generation itself is 12× faster.

Summary. Our evaluation shows that SubSpec is efficient (21 minutes on Internet2), scales to large networks (81 minutes on 500 routers), and is parallelly scalable (8× faster from 1 to 20 threads).

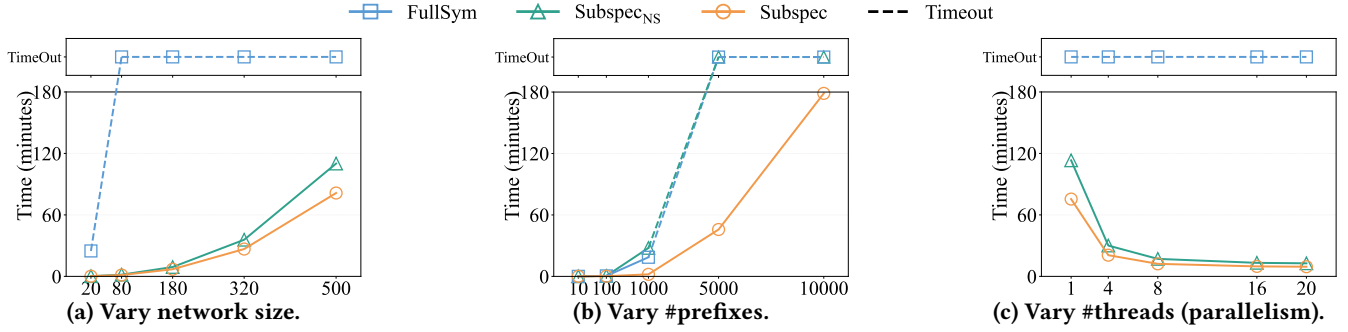


Figure 9: Scalability of subspecification generation under different dimensions.

8 RELATED WORK

A workshop paper [9] introduced localized subspecifications for explaining synthesized network configurations. In contrast, we introduce (i) a formal subspecification formulation, (ii) an efficient generation algorithm with soundness guarantees, and (iii) a user study and large-scale evaluation.

Network verification. Minesweeper [10] introduced symbolic encodings for control-plane verification, offering generality but facing scalability challenges. Subsequent systems such as Plankton [25] and Tiramisu [5] improve scalability through control-plane simulation, model checking, and domain-specific optimizations. Lightyear [28] and Timepiece [6] further scale verification via modular decomposition.

Our work builds on these verification encodings and simulation techniques, but targets *explanation* rather than verification. While modular verification [6, 28] also decomposes global reasoning, it must summarize *all* possible inputs, requiring non-trivial manual specifications. In contrast, localized subspecifications operate under a fixed-input, fixed-context assumption, amenable to automatic generation.

Network synthesis and repair. Propane [11, 12], NetComplete [18], and subsequent synthesis frameworks [4, 17, 26] automatically generate or repair configurations from high-level intents. These approaches typically operate in a black-box manner, offering little insight into why each configuration element is generated as is. Our work complements these systems with explanation for their output configuration.

Differential analysis. Prior work analyzes network differences upon configuration updates. DNA [33] detects property changes via data-plane simulation. Rela [31] checks that intended path or flow changes occur without affecting others. These approaches validate *end-to-end behavioral differences*, whereas our work explains a configuration at a *fine-grained, per-element* level, with explanations expressed as the set of admissible alternative configurations.

Campion [29] also adopts localization to identify deviating sub-components between two configurations, e.g., router

backups or vendor changes. We instead explain a single configuration against its stable state, complementing such analyses by clarifying deviating components and providing semantic guidance for fixes.

Intent inference. Config2Spec [13] and Anime [21] infer *global* specifications from configurations. In contrast, we derive fine-grained *localized subspecifications* and compute them efficiently via router-level decomposition.

Network Debugging. NetCov [30] finds config lines exercised by tests. Network provenance [16, 35, 36] explains *why* behavior occurs by tracing derivations (positive explanations). In contrast, localized subspecifications give *normative, counterfactual* explanations: what each configuration element must satisfy to preserve a property, supporting safe edits.

Explainability for programs. Explainable program analysis and synthesis aim to make automated reasoning more interpretable. Zhang et al. [34] expose internal synthesis steps, while Nazari et al. [23] localize specifications to program structures. Our work shares the same localization principle, but handles distributed protocol convergence and cross-device dependencies absent in single-program settings.

9 CONCLUSION

We present localized subspecifications as a practical way to explain network configurations by exposing the precise conditions each element must satisfy to preserve global properties. Using router-level decomposition and localized symbolic simplification, subspecifications can be generated efficiently and significantly improve maintenance accuracy and efficiency in a user study.

Localization and decomposition involve tradeoffs: localized subspecifications are not compositional, and explain configuration elements with respect to the converged stable state, rather than all alternative or transient states. This focus enables scalability and clear explanations that are difficult to obtain with global reasoning. Future work will explore multi-location reasoning and tighter integration with automated repair and AI-assisted workflows.

REFERENCES

- [1] Deepseek. <https://chat.deepseek.com/>, 2025.
- [2] Doubao. <https://www.doubao.com>, 2025.
- [3] Google gemini. <https://gemini.google.com>, 2025.
- [4] Anubhavnidhi Abhashkumar, Aaron Gember-Jacobson, and Aditya Akella. Aed: Incrementally synthesizing policy-compliant and manageable configurations. In *Proceedings of the 16th International Conference on emerging Networking EXperiments and Technologies*, pages 482–495, 2020.
- [5] Anubhavnidhi Abhashkumar, Aaron Gember-Jacobson, and Aditya Akella. Tiramisu: Fast multilayer network verification. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 201–219, 2020.
- [6] Timothy Alberdingk Thijm, Ryan Beckett, Aarti Gupta, and David Walker. Modular control plane verification via temporal invariants. *Proceedings of the ACM on Programming Languages*, 7(PLDI):50–75, 2023.
- [7] Anonymous Authors. Artifact. <https://anonymous.4open.science/r/Explainable-Network-Verification-via-Localized-Subspecification-B2DF>, 2026. Artifact repository (anonymous submission).
- [8] Mina Tahmasbi Arashloo, Ryan Beckett, and Rachit Agarwal. Formal methods for network performance analysis. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 645–661, 2023.
- [9] Anonymous Authors. Anonymized workshop paper. 2024.
- [10] Ryan Beckett, Aarti Gupta, Ratul Mahajan, and David Walker. A general approach to network configuration verification. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, pages 155–168, 2017.
- [11] Ryan Beckett, Ratul Mahajan, Todd Millstein, Jitendra Padhye, and David Walker. Don’t mind the gap: Bridging network-wide objectives and device-level configurations. In *Proceedings of the 2016 ACM SIGCOMM Conference*, pages 328–341, 2016.
- [12] Ryan Beckett, Ratul Mahajan, Todd Millstein, Jitendra Padhye, and David Walker. Network configuration synthesis with abstract topologies. In *Proceedings of the 38th ACM SIGPLAN conference on programming language design and implementation*, pages 437–451, 2017.
- [13] Rüdiger Birkner, Dana Drachsler-Cohen, Laurent Vanbever, and Martin Vechev. {Config2Spec}: Mining network specifications from network configurations. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 969–984, 2020.
- [14] John Brooke. Sus: a “quick and dirty” usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester and A. L. McClelland (eds.), *Usability Evaluation in Industry*. London: Taylor & Francis, 1996, 1996. Retrieved from Wikipedia page “System usability scale” on 3 Sep 2025.
- [15] Matt Brown, Ari Fogel, Daniel Halperin, Victor Heorhiadi, Ratul Mahajan, and Todd Millstein. Lessons from the evolution of the batfish configuration analysis tool. In *Proceedings of the ACM SIGCOMM 2023 Conference*, pages 122–135, 2023.
- [16] Ang Chen, Yang Wu, Andreas Haeberlen, Wenchao Zhou, and Boon Thau Loo. The good, the bad, and the differences: Better network diagnostics with differential provenance. In *Proceedings of the 2016 ACM SIGCOMM Conference*, pages 115–128, 2016.
- [17] Ahmed El-Hassany, Petar Tsankov, Laurent Vanbever, and Martin Vechev. Network-wide configuration synthesis. In *Computer Aided Verification: 29th International Conference, CAV 2017, Heidelberg, Germany, July 24–28, 2017, Proceedings, Part II 30*, pages 261–281. Springer, 2017.
- [18] Ahmed El-Hassany, Petar Tsankov, Laurent Vanbever, and Martin Vechev. {NetComplete}: Practical {Network-Wide} configuration synthesis with autocompletion. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 579–594, 2018.
- [19] Ari Fogel, Stanley Fung, Luis Pedrosa, Meg Walraed-Sullivan, Ramesh Govindan, Ratul Mahajan, and Todd Millstein. A general approach to network configuration analysis. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, pages 469–483, 2015.
- [20] Nick Giannarakis, Devon Loehr, Ryan Beckett, and David Walker. Nv: An intermediate language for verification of network control planes. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 958–973, 2020.
- [21] Ali Kheradmand. Automatic inference of high-level network intents by mining forwarding patterns. In *Proceedings of the Symposium on SDN Research*, pages 27–33, 2020.
- [22] Xu Liu, Peng Zhang, Anubhavnidhi Abhashkumar, Jiawei Chen, and Weirong Jiang. Automatic configuration repair. In *Proceedings of the 23rd ACM Workshop on Hot Topics in Networks*, pages 213–220, 2024.
- [23] Amirmohammad Nazari, Yifei Huang, Roopsha Samanta, Arjun Radhakrishna, and Mukund Raghothaman. Explainable program synthesis by localizing specifications. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA2):2171–2195, 2023.
- [24] ETH Zürich Networked Systems Group. Config2spec: Mining network specifications from network configurations. <https://github.com/nsg-ethz/config2spec>. Accessed: 2026-01-25.
- [25] Santhosh Prabhu, Kuan Yen Chou, Ali Kheradmand, Brighten Godfrey, and Matthew Caesar. Plankton: Scalable network configuration verification through model checking. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 953–967, 2020.
- [26] Sivaramakrishnan Ramanathan, Ying Zhang, Mohab Gawish, Yogesh Mundada, Zhaodong Wang, Sangki Yun, Eric Lippert, Walid Taha, Minlan Yu, and Jelena Mirkovic. Practical intent-driven routing configuration synthesis. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 629–644, 2023.
- [27] Roland Schmid, Tibor Schneider, Georgia Fragkouli, and Laurent Vanbever. Transient forwarding anomalies and how to find them. *Proceedings of the ACM on Networking*, 3(CoNEXT2):1–23, 2025.
- [28] Alan Tang, Ryan Beckett, Steven Benaloh, Karthick Jayaraman, Tejas Patil, Todd Millstein, and George Varghese. Lightyear: Using modularity to scale bgp control plane verification. In *Proceedings of the ACM SIGCOMM 2023 Conference*, pages 94–107, 2023.
- [29] Alan Tang, Siva Kesava Reddy Kakarla, Ryan Beckett, Ennan Zhai, Matt Brown, Todd Millstein, Yuval Tamir, and George Varghese. Champion: debugging router configuration differences. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, pages 748–761, 2021.
- [30] Xieyang Xu, Weixin Deng, Ryan Beckett, Ratul Mahajan, and David Walker. Test coverage for network configurations. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 1717–1732, 2023.
- [31] Xieyang Xu, Yifei Yuan, Zachary Kincaid, Arvind Krishnamurthy, Ratul Mahajan, David Walker, and Ennan Zhai. Relational network verification. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 213–227, 2024.
- [32] Fangdan Ye, Da Yu, Ennan Zhai, Hongqiang Harry Liu, Bingchuan Tian, Qiaobo Ye, Chunsheng Wang, Xin Wu, Tianchen Guo, Cheng Jin, et al. Accuracy, scalability, coverage: A practical configuration verifier on a global wan. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, pages 599–614, 2020.

- [33] Peng Zhang, Aaron Gember-Jacobson, Yueshang Zuo, Yuhao Huang, Xu Liu, and Hao Li. Differential network analysis. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 601–615, 2022.
- [34] Tianyi Zhang, Zhiyang Chen, Yuanli Zhu, Priyan Vaithilingam, Xinyu Wang, and Elena L Glassman. Interpretable program synthesis. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pages 1–16, 2021.
- [35] Wenchao Zhou, Qiong Fei, Arjun Narayan, Andreas Haeberlen, Boon Thau Loo, and Micah Sherr. Secure network provenance. In *Proceedings of the twenty-third ACM symposium on operating systems principles*, pages 295–310, 2011.
- [36] Wenchao Zhou, Micah Sherr, Tao Tao, Xiaozhou Li, Boon Thau Loo, and Yun Mao. Efficient querying and maintenance of network provenance at internet-scale. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, pages 615–626, 2010.

A IMPLEMENTATION DETAILS

A.1 Seed subspecification generation

We describe in more details the generation of seed subspecification at each router, which is the basis for field-level and line-level subspecifications, and how to handle routing loops.

Encoding of Router-level subspecification. We reuse Minesweeper’s symbolic encoding of router configuration semantics and augment it with the stable routes obtained from Batfish simulation.

Best-route-based encoding scheme. The router-level encoding models:

- the best routes of all peers;
- the export routing policies of peers connected to the local router;
- all import routing policies of the local router;
- the local best-route selection process; and
- the best route of the local router.

Rather than explicitly encoding all per-interface import/export route transformations (Section 5), we encode only neighbors’ best routes and capture import/export behavior implicitly through the corresponding filter and policy logic. This design is motivated by two considerations.

First, it captures the effect of non-transitive and policy-modified attributes (e.g., local preference, MED) on route selection. These attributes affect the local best-route decision only through the interaction among neighbors’ exported routes, the local router’s import policies, and competing alternatives. Encoding neighbor best routes together with import/export policies is therefore sufficient to model their impact. Directly fixing their concrete values would over-constrain the model and yield an overly restrictive router-level subspecification.

Second, this abstraction aligns well with both simulated and real deployments. The only external input required is each router’s best route in the stable state, which is readily available from control-plane simulation or operational state, without modifying existing tools.

Loop prevention. While BGP route loop prevention is already enforced by the underlying symbolic encoding, which models AS-path-based semantics, when we construct router-level subspecifications, we summarize the stable state using only best routes and abstract route-propagation relationships. If this summary is used naively, it may admit combinations of routes that are individually valid but collectively inconsistent with loop-prevention semantics.

To preserve consistency with the stable state, we also record, for each best route, the neighbor from which it is

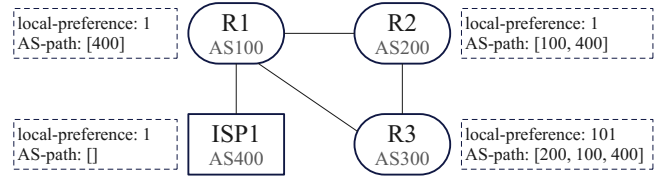


Figure 10: A example for BGP routing loop prevention.

learned (*i.e.*, the route-propagation direction). Figure 10 illustrates this: R3’s best route is learned from R2, so forwarding from R2 to R3 is enabled, while forwarding from R1 to R3 is disabled. This prevents R3 from re-accepting a route that would contain its own AS in the AS-path and thus avoids a routing loop.

We therefore do not re-implement loop prevention; rather, we ensure that the router-level abstraction respects the protocol semantics already captured by the verifier, keeping subspecification generation aligned with the original stable routing behavior.

A.2 Aligning simulation and symbolic encoding

During implementation, we identified and resolved several modeling inconsistencies between the Batfish simulator [19] (version: commit 6128b04, March 2021) and the Minesweeper verifier [10] (version: commit 6128b04, March 2021).

We cross-validated the two models as follows. For each router and prefix, we take the stable best routes computed by Batfish and constrain Minesweeper’s symbolic encoding with these routes. If the resulting constraints are satisfiable, we treat the two tools as behaviorally consistent for that scenario and proceed with subspecification generation.

Using this approach, we identified the following inconsistencies.

BGP best route selection process — AS-path string comparison.

Batfish performs AS-path string comparison as a final step among candidate routes whose key attributes (local preference, AS-path length, MED, etc.) are identical. Minesweeper’s best-route selection process does not include this step. To align the behaviors of Batfish and Minesweeper, we enabled multipath-relax in all Cisco configurations, effectively disabling AS-path string comparison in Batfish. With this adjustment, both tools produce consistent best-route selection results.

BGP best route selection process — tiebreaker.

After comparing multiple route attributes, Batfish may encounter multiple equally preferred candidate routes. When ECMP is not enabled, Batfish enters the tiebreaker stage. In this stage, the Batfish implementation first considers the arrival times of the routes, selecting the earliest-arrived route as the best route. If multiple routes still remain, it then compares the lowest BGP

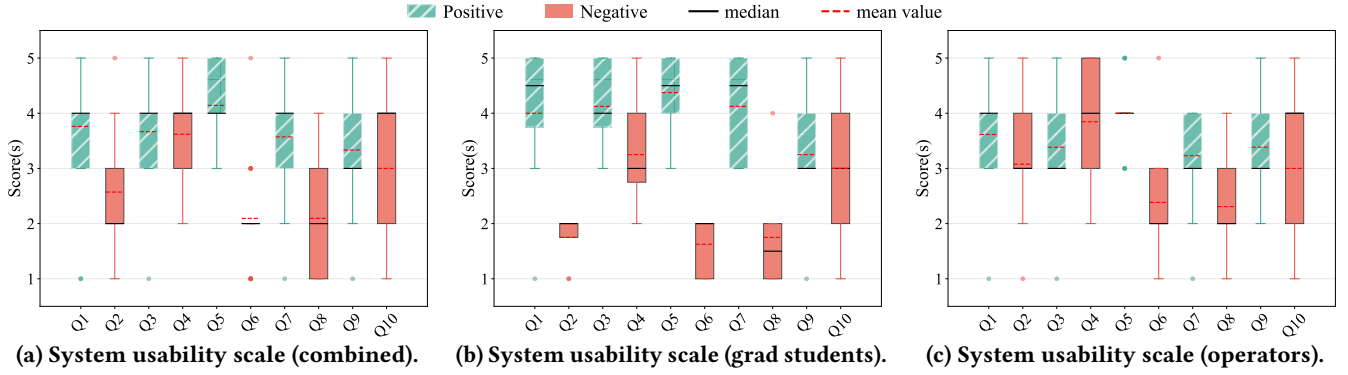


Figure 11: User study system usability scale (SUS).

router ID. In contrast, Minesweeper directly selects the route with the lowest router ID. To align the behavior of the two tools, we temporarily disabled the arrival-time comparison in Batfish.

BGP best route selection process — withdraw and resend. In BGP, when a neighbor’s best route changes, it may first withdraw the old route and then advertise a new one, temporarily replacing routes with the same next hop. We identified an issue in Batfish: when receiving a new route with the same next hop and similar priority as the current best route, Batfish retained the old route and ignored the new one. If the old route was later withdrawn, the router might fail to install the new route and temporarily lose valid routing information. To address this issue, we modified the route selection logic to always prefer a new route from the same next hop, regardless of priority. This ensures correct route installation during withdraw-and-resend events.

BGP routing policies — set community. In BGP, communities are optional transitive attributes used to tag routes and influence routing decisions across autonomous systems. The `set community` command assigns a new set of community values to a route and removes all previously attached communities unless otherwise specified. We identified an issue in Minesweeper: for configurations using `set community`, Minesweeper did not clear existing communities. We corrected this issue to ensure that all non-selected communities are properly removed.

BGP route transfer process — non-transitive attributes. For non-transitive attributes in eBGP, modifications applied by export and import routing policies (e.g., local preference and MED) only affect local best-route selection. These attributes are reset to their default values before routes enter export routing policies. We identified an issue in Minesweeper: it did not reset such attributes before route export in eBGP. We corrected this issue to ensure that these attributes are properly reset during route transfer.

B SYSTEM USABILITY SCALE (SUS) QUESTIONNAIRE AND RESULTS

We include the System Usability Scale (SUS) questionnaire [14] used in our user study. Participants respond on a 5-point Likert scale (1 = Strongly disagree, 5 = Strongly agree).

- (1) I think that I would like to use this system frequently.
- (2) I found the system unnecessarily complex.
- (3) I thought the system was easy to use.
- (4) I think that I would need the support of a technical person to use this system.
- (5) I found the various functions in this system were well integrated.
- (6) I thought there was too much inconsistency in this system.
- (7) I would imagine that most people would learn to use this system very quickly.
- (8) I found the system very cumbersome to use.
- (9) I felt very confident using the system.
- (10) I needed to learn a lot of things before I could get going with this system.

SUS score computation. Following standard SUS scoring [14], for positive items (question 1,3,5,7,9) we subtract 1 from the response, and for negative items (question 2,4,6,8,10) we subtract the response from 5. The adjusted scores are summed and multiplied by 2.5, yielding a final score in the range 0–100.

Overall results. Figure 11 shows the per-question distribution and the breakdown by graduate students and network operators. The system achieved an average SUS score of **62**.

Overall, participants gave a generally favorable usability assessment, suggesting that subspecification annotations fit into their workflow without substantial cognitive overhead. While the score does not fall into the range typically labeled “excellent,” it reflects a moderately positive usability level. Lower ratings on perceived usage barriers (Q4) and learning effort (Q10) contributed most to the overall score.

We next show detailed analysis for each question group.

Adoption intention and perceived complexity(Q1–Q2)

Half of the participants(6/8 graduate students and 10/15 network operators) reported that they would be willing to use our system for tasks such as network configuration changes. Only two participants indicated that they would not use the system, primarily citing the subspec hints as overly complex.

The two groups differed in their perception of complexity. No graduate students viewed the system as complex, whereas only 4/15 engineers reported that it was not complex. This suggests that frontline operators are more likely to perceive subspec hints as requiring additional understanding effort or introducing information overhead. This difference reflects the conservative, risk-aware nature of operational work. It also helps explain why adoption intention remains strong, even though the SUS score is only moderate.

Ease of use and support needs(Q3–Q4) Among graduate students, 6/8 found the system easy to use, compared with 6/15 engineers, indicating differences in intuitive understanding and operational proficiency. Q4 further supports this: 9/15 engineers reported needing technical support, versus 3/8 graduate students. These results suggest that the main barrier may lie in understanding the subspec concept and its application in operational workflows.

Integration and consistency(Q5–Q6) Responses in this dimension were the most stable and positive. 18/23 participants agreed that the system's functions were well integrated, and Q6 responses were largely positive, with only one participant reporting inconsistency. These results indicate broad recognition of the system's coherence and consistency of information presentation.

Learnability and perceived effort(Q7–Q8) Graduate students reported higher learnability than engineers (5/8 vs. 7/15), suggesting that conceptual understanding, rather than interaction burden, is the primary learning challenge. Consistently, 7/8 graduate students found the system not cumbersome, compared with 9/15 engineers, indicating that conceptual overhead disproportionately affects engineers.

User confidence and learning cost(Q9–Q10) The confidence levels were similar between the groups (3/8 graduate students, 6/15 engineers). Both groups reported that some learning was required (4/8 graduate students, 8/15 engineers), with engineers slightly more likely to perceive a higher learning burden, consistent with Q4 and reflecting the cost of understanding the subspec concept.

Summary The main strengths of the system lie in well-integrated functions, a consistent interface, and a low operational burden. Limitations in overall usability are primarily due to the learning cost introduced by the subspec concept

and engineers' heightened sensitivity to complexity and support needs.

In particular, engineers' cautious evaluations likely reflect the risk constraints of real production environments: configuration changes carry high failure costs, and operational practice emphasizes explainability, controllability, and rollback capability, making them more conservative toward adopting new concepts and workflows. These findings also explain why, despite moderately positive SUS scores, a relatively high proportion of participants remain willing to use the system in practical tasks.